CASE-STUDY PRACTICE | CAPSTONE 5 / 11 | STUDY EDITION

Harmful Content Detection: Rare Classes, Labels and a Moderation Cascade

Scenario questions on classifying undesired text when the cases that matter are rare, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on data, a model or a serving system: figures are the paper's or arithmetic from stated inputs, and traffic volumes, prices, token counts and model widths are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: taxonomy, rarity, metric and the line with policy 6

2 Data and labels: instructions, audits and active learning 10

3 The model: heads, synthetic data, domain adaptation and probing 15

4 Serving, streaming, evaluation and operations 21

Sources and confidence 27

Every question, by part

Part 1 Framing the problem: taxonomy, rarity, metric and the line with policy 6

01 Why eight labels and not one? Taxonomy 6

02 Price the rarity Estimation 7

03 Why not accuracy, and why not ROC? Metric 8

04 Where does the classifier end? Scope 9

Part 2 Data and labels: instructions, audits and active learning 10

05 Make the instructions followable Labeling 10

06 How do you audit a rare class? Quality 11

07 Mix the selection pipelines Active learning 12

08 Review the selection function Code review 13

Part 3 The model: heads, synthetic data, domain adaptation and probing 15

09 Eight heads on one backbone Architecture 15

10 Patch weak spots with synthetic data Data 16

11 When does domain adversarial training pay? Domain shift 17

12 What is the model really looking at? Probing 18

13 Review the key token finder Code review 19

Part 4 Serving, streaming, evaluation and operations 21

14 Price the streaming cascade Design 21

15 Review the stream gate Code review 22

16 Read the comparison table honestly Evaluation 24

17 What does the paper say its model gets wrong? Limits 25

18 Data consent, annotator welfare and ten times the traffic Operations 26

The case in one page

A summary of the case study to read before the questions.

Problem	Score text against a taxonomy of undesired categories, for the inputs and outputs of language models, when the categories that matter are the rarest and the labels are subjective.
Core idea	A designed taxonomy and labeling process, an active-learning loop that spends labels on rare cases, synthetic data to patch weak spots, and probing to catch a model that keys off the wrong words.
Taxonomy	Five categories, with subcategories; the model predicts 8 labels: S, H, V, SH, HR and the three severe subcategories S3, H2, V2 (source-confirmed).
Rarity	Among sampled user prompts, 0.04% involved self-harm and 0.017% hateful content with threats (source-confirmed).
Headline results	Active learning: 40% of selected samples had an undesired label against 3.4% for random, 10 to 22 times per category. The model beats Perspective API on the four categories they share (source-confirmed).
Surprises	Large amounts of noisy synthetic data hurt. Domain adversarial training helped early and little or negatively with labeled production data (source-confirmed).
First to break	The labeling budget: about 111,000 random labels for 100 self-harm examples against about 5,400 with active learning (illustrative arithmetic on Table 4).
Easy-to-miss traps	AUPRC cannot be ranked across categories. A streaming judge that re-reads all the text costs more than judging once. A short key token is not a shortcut. A 346% gain on one row is not an average.

Numbers to know

Each figure carries the tag that says how it was obtained. Traffic volumes, prices, token counts and model widths are illustrative, because the paper reports rates and not volumes.

Quantity	Value	Evidence
Rarity	0.04% self-harm; 0.017% hateful threats, among sampled user prompts	source-confirmed (the paper)
Daily cases	10,000,000 × 0.0004 = 4,000 self-harm; × 0.00017 = 1,700 hateful threats	illustrative; recomputed
Table 4	SH random 0.09%, active 1.85% (20.6×); H2 22.3×; overall 40% against 3.4%; about 2,000 labels a round, 3 rounds, 500,000 scored	source-confirmed (the paper)
Labels per 100 positives	Random: 100 ÷ 0.0009 = about 111,000; active: 100 ÷ 0.0185 = about 5,400	recomputed
Random audit	10 positives need 10 ÷ 0.0004 = 25,000 random items	recomputed
Rare-class ROC trap	10 million items, 4,000 positives, 80% recall, 1% false positive rate: 99,960 false alarms; precision 3.1%	illustrative; recomputed
Review load	Precision 3.1%: about 32 reviews per real case; at 2 minutes, 64 minutes	illustrative; recomputed
Training data	PUB about 90,000; SYN adds 69,000; MIX adds 60,000 labeled production samples	source-confirmed (the paper)
Synthetic data	69,000 curated examples raised hateful-content AUPRC from 0.417 to 0.551; noisy synthetic data at twice the labeled size hurt	source-confirmed (the paper)
Key token probing	722.3 to 15.9 characters; about 97% of key tokens unsafe; all immigrants.: 17 of 21 examples undesired (81%)	source-confirmed; recomputed
Table 5 means	PUB .385 to .532; SYN .458 to .533; MIX .713 to .728	source-confirmed (table); recomputed
Table 3	Public set S .9703/.8709, H .7968/.6914, V .7371/.5201, HR .6191/.3902; Perspective wins Jigsaw insult, obscene, threat, toxic	source-confirmed (the paper)
Judge cost	700 + 20 tokens at 3 and 15 dollars per million: 0.0024 dollars; 50,000 a day: 120 dollars; 8% only: 9.60 dollars	illustrative; recomputed
Tier 1 re-scoring	500 tokens checked every 3: 166 checks, about 41,600 tokens (83×); total with 8% judged 71.97 dollars (40% saving)	illustrative; recomputed
Windowed tier 1	64 tokens every 3: 10,624 tokens; total 25.54 dollars (79%); every 50 tokens: 13.73 dollars (89%)	illustrative; recomputed
Head parameters	d_model 768: 196,864 per head, 1,574,912 for 8; one 256-unit MLP with 8 outputs 198,656	illustrative; recomputed
Key token cost	$n^2/2$ calls: 180 tokens, 16,290 calls per text	recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Label set	8 labels, with three severe subcategories	Fewer labels if the policy has fewer categories	source-confirmed
Data collection	Active learning over production traffic	Random sampling if every category is common	source-confirmed
Audit sampling	Stratified, scored by F1 per category	Random if all categories are frequent	source-confirmed
Model	Fine-tuned classifier with one head per label	A prompted LLM if the taxonomy changes often	source-confirmed; reasoned
Domain adaptation	Domain adversarial training when labeled production data is scarce	Skip it once in-distribution labels are plentiful	source-confirmed
Streaming checks	Windowed or sparse tier 1, judge on the uncertain band	Whole-text scoring only for short outputs	reasoned
Enforcement	A separate policy layer	A fixed threshold for one product and one policy	source-confirmed; reasoned
Metric	AUPRC per category, compared with a chance baseline	Accuracy or ROC only when classes are balanced	source-confirmed; reasoned

PART 1

Framing the problem: taxonomy, rarity, metric and the line with policy

Defining what counts as undesired, pricing the rarity of the cases that matter, choosing a metric that survives class imbalance, and deciding where the classifier stops.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Taxonomy	A structured set of categories and subcategories that defines what kinds of content count as a violation. Labeling, training and evaluation all depend on it.
Multi-label classification	Each item can have several labels at once. The model gives an independent score per label, because a text can be both violent and hateful.
Class imbalance and prevalence	Prevalence is the share of items in a class. When a class is very rare, accuracy and ROC curves look good for useless models.
Precision and recall	Precision is the share of flagged items that are truly positive. Recall is the share of true positives that were flagged.
AUPRC	Area under the precision-recall curve. It averages precision across all recall levels. A model that guesses at random scores the prevalence of the class.
ROC curve and AUC	The curve of true positive rate against false positive rate. The false positive rate divides by the number of negatives, so with millions of negatives a tiny rate is still a huge number of false alarms.
Threshold and policy layer	The classifier gives a score. A separate layer turns a score into an action such as allow, block or send to a reviewer, using a threshold chosen for each policy and product.
Moderation endpoint	A service that returns category scores for a piece of text. The paper's model is available through one and only computes scores.
Where to read	Markov et al.: A Holistic Approach to Undesired Content Detection in the Real World (arXiv:2208.03274, AAAI 2023)

01 Why eight labels and not one?

TAXONOMY

SCENARIO

A product manager wants a single score called undesired. The paper has five categories and subcategories, and its model predicts 8 labels. Argue for the taxonomy and say what it costs.

HOW TO THINK ABOUT IT

Look at what a single bucket would blur, then at what the subcategories do for the people who label the data.

Q What does the model predict?

A Eight labels: the five top categories S, H, V, SH and HR, and the three most severe subcategories S3, H2 and V2 (source-confirmed).

- Each label is its own score, since a text can be violent and hateful at once.

Q What would one bucket lose?

A Severity. The paper's example: kill all members of a group has different repercussions from a group are dishonest. Both would be undesired, and they call for different handling.

- It also loses the not-undesired cases, such as medical sexual content (S0) and a quote with commentary (H0.b).

Q What do subcategories do for labeling?

A They reduce the disagreement rate and corner cases during labeling, because each subcategory is designed to be as mutually exclusive as possible (source-confirmed).

- A named, narrow definition is easier to apply the same way each time than a broad one.

Q What does context do to the label?

A It can change it. I will kill you would be labeled violence and may be fine in fiction, as the paper says. A taxonomy is a starting point that each use evaluates for itself.

- The paper's S2 maps most closely to what is illegal in the USA, so even the definitions carry a jurisdiction.

Q What does it cost?

A More labels to maintain, more labeling instructions, more positive examples needed per rare subcategory, and more thresholds to set.

- The paper keeps the model to 8 labels, not every subcategory, to hold that cost down.

Q Reframe it.

A Design the label set for the people who label it and for the decisions it feeds. Granularity buys consistency and severity, and costs data per label.

Basis: The 8 labels, the severity example, the effect on disagreement and the context point are from the paper (source-confirmed); the cost list is reasoned.

02 Price the rarity

ESTIMATION

SCENARIO

The paper says that among sampled user prompts only 0.04% included self-harm and 0.017% included hateful content with threats. Your team will label 2,000 items a round. Plan the data collection, using the paper's own Table 4 rates for random and active selection.

HOW TO THINK ABOUT IT

Compute how many positives each approach finds in a batch, then how many labels 100 positives would take.

Q What does random sampling give you?

A At the production rate of 0.04%, 2,000 random items hold 0.8 self-harm examples on average. In the paper's random arm the rate was 0.09%, so 1.8 (source-confirmed, Table 4).

- That is nowhere near enough for a model to learn the category.

Q What does active selection give?

A The paper's active strategy produced 1.85% self-harm among selected samples, so 37 in a batch of 2,000, which is 20.6 times as many.

- The multiplier depends on how the three selection pipelines were mixed and on the thresholds, which the paper does not report, so it is a property of that setup.

Q How many labels for 100 positives?

A Random: 100 ÷ 0.0009 = about 111,000 labels. Active: 100 ÷ 0.0185 = about 5,400 labels.

- At an assumed cost per label the difference is the whole budget. The binding constraint is labeled positives, not compute.

Q Is rare the same as negligible?

A No. With an assumed 10 million submissions a day, 0.04% is 4,000 self-harm cases a day, and 0.017% is 1,700 hateful threats (illustrative).

- Rare as a share and large in absolute count at the same time.

Q Reframe it.

A Convert a rate into labels per positive. The cost of data for a rare class is set by how many items you must look at to find one.

Basis: The 0.04%, 0.017% and Table 4 rates are from the paper (source-confirmed); the 10 million submissions a day is illustrative; arithmetic recomputed.

03 Why not accuracy, and why not ROC?

METRIC

SCENARIO

A candidate says: our self-harm model has 99.9% accuracy and a ROC curve that reaches 80% true positive rate at a 1% false positive rate, so it is excellent. The paper reports AUPRC. Show what the numbers hide.

HOW TO THINK ABOUT IT

Use 10 million items with 0.04% positives. Compute the false alarms at that operating point, then the precision.

Q What does accuracy hide?

A A model that flags nothing is right on 99.96% of items. Accuracy is dominated by the negatives, which is what the paper means by an accuracy that can be arbitrarily high due to the abundance of true negatives (source-confirmed).

- It would score above 99.9% while finding no case.

Q What does that ROC point mean?

A Positives: 4,000. True positives at 80%: 3,200. Negatives: 9,996,000. False positives at 1%: 99,960.

- The model raises about 103,000 alerts for 3,200 real cases.

Q What is the precision?

A 3,200 ÷ (3,200 + 99,960) = 3.1%. About 32 alerts for each real case.

- The ROC curve looks excellent because its false positive rate divides by 10 million negatives. Precision-recall puts the burden on the positive class.

Q What does AUPRC compare against?

A A random scorer scores the prevalence, here 0.0004. So an AUPRC of 0.8 is far above chance, and an AUPRC depends on how rare the category is.

- That makes AUPRC values from categories with different prevalence not comparable with each other, only with their own baseline and with another model on the same set.

Q Reframe it.

A Pick the metric whose denominator is the class you care about, and compare against its chance level. A curve that looks good can hide a flood of false alarms.

Basis: The statement about accuracy is from the paper (source-confirmed); the 10 million items, 80% and 1% are illustrative arithmetic (recomputed); the AUPRC and ROC reasoning is standard and the paper does not give a reason for its choice.

04 Where does the classifier end?

SCOPE

SCENARIO

Leadership asks the team to ship content moderation and expects the model to decide what is removed. The paper builds a classifier and says it should be paired with fair and robust enforcement practices. Draw the line and show why it helps.

HOW TO THINK ABOUT IT

Ask what changes between products and policies, and what a threshold costs in reviewer time.

Q What does the paper build?

A A scorer. The deployed model only computes safety scores and cannot generate text (source-confirmed). Enforcement practices are named as a separate, paired thing.

- Block, remove, demote or escalate are decisions outside the model.

Q Why separate them?

A The same scores can feed different thresholds for different policies, regions and surfaces without retraining. A children's product and a research tool want different cut-offs.

- The model's job is to be calibrated across categories, and the policy decides what each score means in use.

Q What does a threshold cost?

A Take the precision of 3.1% from Question 3. Each real case costs about 32 reviews. At an assumed 2 minutes a review, that is about 64 minutes of review per real case.

- A stricter threshold with precision 50% costs 2 reviews, 4 minutes, and misses more cases. The right point is a policy decision.

Q What goes in the policy layer?

A Per-category thresholds, actions by severity, an escalation route to people, appeals, and logging for audit.

- Treat the layer as a product with owners. The paper describes none of it.

Q Reframe it.

A Keep prediction apart from decision. A score is an input, and the threshold encodes costs and values that belong to the people who own the policy.

Basis: The scorer-only statement and the pairing with enforcement are from the paper (source-confirmed); the 2 minutes a review and the 50% operating point are illustrative; arithmetic recomputed.

PART 2

Data and labels: instructions, audits and active learning

Writing labeling instructions people can follow the same way, auditing when positives are rare, and spending a labeling budget on the examples that teach the most.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Labeling instructions	The written definitions and examples annotators follow. Vague instructions make annotators rely on judgment, and the labels become inconsistent.
Calibration session	A regular meeting to review ambiguous cases and disagreements between annotators and auditors, after which the instructions are made clearer.
Audit	Re-checking a sample of labels with a more expert reviewer. For rare classes the sample must be chosen, not random.
F1 score	The harmonic mean of precision and recall. The paper scores the annotators' labels against the auditors' labels with it.
Active learning	Letting the pipeline choose which unlabeled items humans label next, so that each label teaches more.
Uncertainty sampling	Selecting items the model is least sure about, with a score closest to 0.5.
Cold start	The stage where there are no labeled examples of a rare category yet, so a first model cannot be trained from real data.
Mislabel detection by cross-scoring	Splitting the data in two, training a model on each half, and flagging items where the model that did not see an item disagrees with its label.
Where to read	Markov et al.: A Holistic Approach to Undesired Content Detection in the Real World (arXiv:2208.03274, AAAI 2023)

05 Make the instructions followable

LABELING

SCENARIO

Annotators disagree on borderline cases, and the model trained on their labels is erratic. The paper says that detailed definitions, mutually exclusive categories and calibration sessions help. It also discusses disagreement in a more nuanced way. Describe the process and its limits.

HOW TO THINK ABOUT IT

Follow the paper's steps, then read what it says about disagreement as a signal.

Q What are the steps?

A Write detailed, concrete definitions. Design categories and subcategories as mutually exclusive as possible. Hold regular calibration sessions on edge cases and on disagreements between external annotators and internal auditors, and add examples and clearer definitions of borderline cases (source-confirmed).

- As rules become concrete, annotators apply them more consistently.

Q Why does this matter for the model?

A The paper says instructions that lack precision force annotators to rely on subjective judgment, and the inconsistent labels confuse the model.

- Label noise limits what any model can learn.

Q What does the paper say about disagreement?

A Disagreement can be a meaningful signal, especially for data about marginalized groups, and resolving it by majority vote can erase minority views. Annotators' backgrounds shape what they see as hate (source-confirmed).

- The paper presents this as context. It does not say that it preserved disagreements in its own data.

Q What would you do with both?

A Use calibration to remove avoidable ambiguity, keep the annotators' separate votes for hard cases, and track items where trained annotators still disagree.

- Those items show where the taxonomy is unclear or where views differ, and a single hard label hides it.

Q Reframe it.

A Clear rules reduce noise from careless judgment. Some disagreement is information, so measure it before you erase it.

Basis: The process and the discussion of disagreement are from the paper (source-confirmed); the suggestion to keep separate votes is reasoned.

06 How do you audit a rare class?

QUALITY

SCENARIO

An auditor proposes checking 1,000 randomly chosen labeled items per week and reporting the agreement rate. The paper does something different. Explain why the proposal fails for self-harm, and describe the paper's audit.

HOW TO THINK ABOUT IT

Compute how many random items you must audit to see a few positives, then read the paper's design.

Q What does a random audit see?

A At 0.04% prevalence, 1,000 random items hold 0.4 positives. To see 10 you would audit 10 ÷ 0.0004 = 25,000 items.

- The sample is almost all safe content, and says nothing about whether rare labels are right.

Q What does agreement rate measure here?

A Almost nothing. Annotators and auditors will agree on the safe majority, and the paper says the rate can be arbitrarily high due to the abundance of true negatives (source-confirmed).

- Accuracy is the wrong metric for the same reason as in Question 3.

Q What does the paper audit?

A Per category, 10 samples labeled undesired and 10 samples with model probability above 50%. The first group finds false positives and the second estimates recall. It scores the annotators' labels with F1 against the auditors' labels.

- Metrics are kept per category, so category-specific problems stand out and the annotators can be retrained.

Q What does this audit not give?

A An unbiased estimate of overall label accuracy, since the sample is chosen. The paper accepts that, because that number would be uninformative anyway.

- It also depends on the model for half of the sample, so a model that is blind to a pattern will not surface its mistakes there.

Q Reframe it.

A Sample the audit where the errors are. With rare classes, a chosen sample says more per item than a random one, and gives up a global estimate you did not need.

Basis: The audit design and the statement about accuracy are from the paper (source-confirmed); the 1,000 items a week is illustrative; arithmetic recomputed; the limitation is reasoned.

07 Mix the selection pipelines

ACTIVE LEARNING

SCENARIO

Your first model flags few self-harm items. A colleague proposes labeling only the items with the highest scores each round, since those are most likely to be positive. The paper uses three pipelines in parallel. Explain why.

HOW TO THINK ABOUT IT

Ask what a top-score-only labeled set can never contain, then read what each of the paper's pipelines adds.

Q What does a top-score-only set miss?

A Every item the model scores low and gets wrong. Those false negatives are never labeled, so the model never learns from them, and nobody can measure its recall.

- The labeled set also drifts away from production, since it contains only what the model already believes.

Q What are the paper's three pipelines?

A Random sampling, so that part of the data stays consistent with production. Random samples above a score threshold per category, to find likely positives. Uncertainty sampling, taking samples with a score closest to 0.5 (source-confirmed).

- The selections are aggregated and reweighted, with weight proportional to the square root of the sample count of a metadata group, to improve diversity.

Q What does the random arm protect?

A An unbiased view of production: prevalence, recall and the effect of changes. Without it, those cannot be estimated from the labeled data.

- The paper says it updates the mix of strategies over time, and calls its strategy simple.

Q What does the experiment show?

A After 3 iterations of 2,000 labels, active selection gave labeled samples where about 40% had an undesired label, against 3.4% for random selection, and improved performance on all categories (source-confirmed).

- The comparison is of two selection strategies on one setup, so the 10 to 22 times is not a general law.

Q Reframe it.

A A learning loop that only reads what it already believes cannot correct itself. Keep an arm that samples the world as it is.

Basis: The pipelines, the reweighting, the 40% and 3.4% figures and the improvement statement are from the paper (source-confirmed); the argument against top-score-only is reasoned.

08 Review the selection function

CODE REVIEW

SCENARIO

A colleague writes the selection step below for each labeling round and says it implements active learning. After three rounds, recall measured on a random test set is worse for a pattern of phrasing the model scored low, and nobody can estimate production prevalence. Find the faults.

```
def select_for_labeling(scored, budget=2000):
    # scored: list of (item, score) for one category, from the latest model
    ranked = sorted(scored, key=lambda pair: -pair[1])
    return [item for item, _ in ranked[:budget]]
```

HOW TO THINK ABOUT IT

Compare the function with the paper's three pipelines and the reweighting.

Q What is the first fault?

A It only takes the highest scores. Items the model scores low never reach a human, so false negatives are invisible.

- Add random and uncertainty selection.

Q What is the second?

A There is no random arm, so the labeled set is not representative of production. Prevalence and recall cannot be estimated from it.

- The paper keeps a random pipeline so that part of the data stays consistent with the underlying distribution.

Q What is the third?

A It ignores metadata. A burst of near-duplicate items from one source or template can fill the whole budget.

- The paper reweights by the square root of each metadata group's count, which lowers the weight of big groups and raises thin ones without flattening to uniform.

Q State the fix.

A Split the budget between random, above-threshold and near-0.5 items, then draw within each by metadata groups with weights proportional to the square root of group size.

- Test: a pattern the model scores low should appear in the sample at about its random rate.

Q Reframe it.

A A selection rule is a statistical design. Ask what it can never select, what it over-selects, and whether you can still estimate what you need from its output.

REFERENCE CODE

```
import math, random
from collections import defaultdict

def select_for_labeling(scored, group_of, budget=2000, mix=(0.4, 0.3, 0.3), thr=0.5):
    random_arm = random.sample(scored, int(budget * mix[0]))
    above = [p for p in scored if p[1] >= thr]
    above_arm = random.sample(above, min(len(above), int(budget * mix[1])))
    uncertain = sorted(scored, key=lambda p: abs(p[1] - 0.5))[: int(budget * mix[2])]
    picked = {id(p): p for p in random_arm + above_arm + uncertain}.values()
    groups = defaultdict(list)
    for p in picked:
        groups[group_of(p[0])].append(p)
    # keep a share of each metadata group proportional to the square root of its size
    total = sum(math.sqrt(len(g)) for g in groups.values())
    out = []
    for g in groups.values():
        k = max(1, round(budget * math.sqrt(len(g)) / total))
        out += random.sample(g, min(k, len(g)))
    return [item for item, _ in out]
```

Basis: The three pipelines and the square-root reweighting are from the paper (source-confirmed); the faults and the fix are reasoned and the fix was checked by hand and not run.

PART 3

The model: heads, synthetic data, domain adaptation and probing

Why one head per label, what synthetic data helps and hurts, when a domain adaptation trick pays, and how to find out what a model is really looking at.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Transfer learning	Starting from a model pretrained on a large general text corpus and fine-tuning it on the task, instead of training from random weights.
MLP head	A small stack of fully connected layers on top of a shared backbone, giving one output score. Here there is one head per label.
Synthetic data	Training examples generated by a language model from templates or prompts and then labeled, used for categories that have little real data.
Counterfactual bias	A model that scores a text differently when only a demographic word changes, for example giving a high hate score to a neutral phrase with an identity term.
Domain shift	Training data that differs from the data the model will see, for example short public tweets against long production prompts.
Domain adversarial training	Training the model so that a critic cannot tell which domain an example came from, which pushes the model toward features that work in both domains.
Key token probing	Removing tokens one at a time, always dropping the least important, until the prediction falls below a threshold. The tokens that remain are what the model relies on.
Red-teaming	People trying on purpose to make the model fail, to find weaknesses a test set does not show.
Where to read	Markov et al.: A Holistic Approach to Undesired Content Detection in the Real World (arXiv:2208.03274, AAAI 2023)

09 Eight heads on one backbone

ARCHITECTURE

SCENARIO

The paper's model is a lightweight transformer decoder initialized from a GPT model, with the output layer replaced by 8 separate MLP heads. A reviewer asks why not one output layer of 8 numbers, and whether the paper's claim of fewer parameters holds.

HOW TO THINK ABOUT IT

State what the paper says, then count parameters under an assumed model width.

Q What is the design?

A A shared transformer decoder with 8 heads, each an independent matrix of shape [d_model, 256, 1], one per label. It starts from a GPT model pretrained on a large text corpus and is fine-tuned with learning rate 0.05, batch size 256, dropout 0.1 in the heads and up to 3 epochs (source-confirmed).

- This is transfer learning: the backbone brings broad language knowledge, and the heads specialize it.

Q Why separate heads?

A The paper says the head design works better than a single deep MLP layer with one output vector of 8 dimensions at avoiding interference between categories, and needs fewer parameters to train.

- Each label gets its own path from the shared representation, so training one label does not push another.

Q Count the parameters.

A Take d_model = 768 (assumed). One head: 768 × 256 + 256 = 196,864. Eight heads: 1,574,912.

- A single 256-unit MLP with an 8-dimensional output: 768 × 256 + 256 × 8 = 198,656, which is far fewer.

Q So is the claim wrong?

A It cannot be checked from the paper. It compares with a deep MLP whose size the paper does not give. The comparison with a single 256-unit layer gives the opposite answer.

- Quote the claim as the paper's, and say that the arithmetic depends on the alternative.

Q Reframe it.

A A claim of fewer parameters needs a named alternative. When the alternative is not given, report the claim as the author's and do not try to prove it.

Basis: The architecture and the claims are from the paper (source-confirmed); the width of 768 is illustrative; arithmetic recomputed.

10 Patch weak spots with synthetic data

DATA

SCENARIO

A rare category has almost no public data, and an early model gives a high hate score to a neutral phrase with an identity word. The paper uses synthetic data in three ways and also found a limit. Plan it.

HOW TO THINK ABOUT IT

Match each use to its problem, then read what happened when the volume went up.

Q What is the cold-start use?

A Zero-shot prompts to GPT-3 built from human-crafted templates, to produce the first dataset for categories such as SH and V2 where public data is hard to find. The generated texts are then labeled (source-confirmed).

- It starts the process: a first model, and annotators who have examples to train on.

Q What is the rare-category use?

A Few-shot prompts built from existing undesired examples, for subcategories with minimal data even after several iterations of active learning. The generated texts are manually inspected to avoid bias amplification.

- Generated text copies the biases of its prompts, so a person checks it.

Q What is the counterfactual use?

A Template data such as [subject] is selfish, with the subject filled by real demographic attributes, which give hateful samples, or random object names such as black blanket, which give safe samples. Adding 69,000 of these raised the average hateful-content AUPRC from 0.417 to 0.551.

- The contrast teaches that a negative word or an identity group alone is not hate, and the two together may be.

Q What was the limit?

A Large amounts of noisy synthetic data hurt. Examples twice the size of the labeled set, labeled by how they were prompted, lowered performance. The paper suspects that noise confuses a model that already has a large high-quality dataset.

- Other work found synthetic data helpful, mostly with very little real data. The paper notes that difference.

Q Reframe it.

A Use generated data to fill specific, known holes, check it, and keep its share small once you have good real data.

Basis: The three uses, the 0.417 to 0.551 result and the limit are from the paper (source-confirmed); the planning advice is reasoned.

11 When does domain adversarial training pay?

DOMAIN SHIFT

SCENARIO

Models trained on public data do poorly on production prompts, which are longer and often contain few-shot prompts. The paper adds domain adversarial training. A teammate says it gave a 346% gain in self-harm and wants it in every model. Read Table 5 before you answer.

HOW TO THINK ABOUT IT

Compute the means over categories and look at which rows went down.

Q What is the method and its setup?

A A domain critic head tries to tell public examples from production ones, and the model is trained against it. The DAT models used 100,000 additional unlabeled production examples, were trained for up to 2 epochs, and each run was repeated 3 times with different seeds (source-confirmed).

- The aim is representations that do not depend on which domain an example came from.

Q What does the table say about the mean?

A Mean AUPRC over the 8 labels: PUB .385 to .532 (+0.147), SYN .458 to .533 (+0.075), MIX .713 to .728 (+0.015).

- The gain shrinks as more in-distribution data is added. The means are arithmetic on the paper's table.

Q What about individual rows?

A The self-harm row went from .063 to .281 on PUB, +346%, and from .621 to .632 on MIX, +1.8%. On MIX, S, H and V fell slightly (.943 to .939, .843 to .818, .640 to .633). On SYN, H fell from .551 to .476.

- The paper says DAT may slightly hurt categories with a sufficient amount of in-distribution data, such as H and V.

Q What does the paper suspect?

A That the model failed to find a representation that works very well for both the public datasets and the production distribution. It calls for further study.

- So it is not an add-on to include everywhere.

Q What would you decide?

A Use it early, when you have labeled public data and unlabeled production data. Re-test per category once labeled production data arrives, and drop it where it hurts.

- The cheapest fix for domain shift is labeled data from the target domain, which active learning supplies.

Q Reframe it.

A A percentage gain on one row is not an average. Look at every row and at the baseline each came from before you generalize.

Basis: Table 5 values, the setup and the paper's remarks are from the paper (source-confirmed); the means are arithmetic (recomputed); the decision rule is reasoned.

12 What is the model really looking at?

PROBING

SCENARIO

The model gives a high hate score to the phrase all immigrants. A researcher says the model has learned hate. The paper investigates exactly this. Work out what is happening and how to detect it at scale.

HOW TO THINK ABOUT IT

Compute what the data says about the phrase, then follow the paper's input reduction method and its results.

Q What does the data say about the phrase?

A The paper found 21 training examples containing the phrase from a public dataset, 17 undesired and 4 safe: 81% positive (source-confirmed).

- The model learned that the phrase predicts hate, which the data supports, and is not a statement about meaning.

Q How does key token probing work?

A Remove tokens greedily until the prediction drops below 0.8, and look at what remains. On average the paper reduced samples from 722.3 to 15.9 characters.

- A short remainder is expected for texts that contain a slur or a threat. What matters is what the remainder is.

Q What did it find?

A About 97% of the identified key tokens were unsafe, checked against third-party APIs and keywords, so the model mostly used the right feature. The other 3%, reviewed by hand, showed three problems.

- Irrelevant symbols such as a period contributing to the score. Long key tokens close to the original example, hinting at memorization. Weakness with typos and adversarial text, for example w h o r e s reducing to e s.

Q What did red-teaming add?

A A high hate score for any text containing the # token, probably from short tweets in academic data, and high hate and violence scores for racial words, so that I hate black cats! was a risk.

- Both are surface features. The paper patched them with synthetic template data and by adjusting the mix of academic text.

Q Reframe it.

A Do not read a high score as understanding. Ask what the model keys off, and test the answer with removal, with edited inputs and with people trying to break it.

Basis: The 17 of 21 examples, the reduction figures, the 97% and 3% findings and the red-teaming results are from the paper (source-confirmed); the 81% is arithmetic (recomputed).

13 Review the key token finder

CODE REVIEW

SCENARIO

A colleague implements input reduction to find what the model keys off. The tokens it returns never trigger the model when you feed them back, and the job takes days on a sample of a few thousand texts. The paper removes tokens greedily until the prediction drops below 0.8. Find the faults.

```
def key_tokens(tokens, score_fn, threshold=0.8):
    kept = list(tokens)
    while len(kept) > 1:
        best = max(range(len(kept)), key=lambda i: score_fn(kept[:i] + kept[i+1:]))
        kept = kept[:best] + kept[best+1:]
        if score_fn(kept) < threshold:
            break
    return kept
```

HOW TO THINK ABOUT IT

Check what is returned when the loop ends, and count calls to the scoring function.

Q What does it return?

A The tokens after the score has fallen below the threshold. By construction they no longer trigger the model, so they are not the key tokens.

- The paper's set is the smallest one that still gives a prediction of at least 0.8.

Q What does it cost?

A Each round scores every candidate removal. For n tokens, round k scores n − k + 1 candidates, so about $n^2/2$ calls. For a 180-token text, 16,290 calls.

- A sample of 90,000 texts at that length would be 1.5 billion calls.

Q What other fault is there?

A It scores the new text again after each removal, which repeats a call it already made when choosing.

- Keep the score from the best candidate.

Q State the fix.

A Remember the last token list whose score is at least the threshold, return that, reuse the chosen candidate's score, and run it on a sample of texts, not the whole set.

- Test: feeding the returned tokens back gives a score of at least 0.8, and removing any one of them drops it below the threshold or close to it.

Q Reframe it.

A Return the last state that satisfied the condition, not the one that broke it. Count the calls of an inner loop before running it over a dataset.

REFERENCE CODE

```
def key_tokens(tokens, score_fn, threshold=0.8):
    kept = list(tokens)
    while len(kept) > 1:
        scored = [(score_fn(kept[:i] + kept[i+1:]), i) for i in range(len(kept))]
        best_score, best = max(scored)
        if best_score < threshold:
            break                       # the next removal would stop the trigger
        kept = kept[:best] + kept[best+1:]
    return kept                         # smallest list still scoring at least threshold
```

Basis: The method is from the paper (source-confirmed); the faults and the 180-token example are reasoned arithmetic (recomputed); the fix was checked by hand and not run.

PART 4

Serving, streaming, evaluation and operations

Moderating a stream with a cheap filter and an expensive judge, reading a comparison table honestly, and running the system responsibly.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Cascade	A pipeline in which a fast, cheap model screens every item and only flagged or uncertain items go to a slower, more accurate model.
LLM as a judge	Using a language model with a rubric to decide whether an output violates a policy. It adapts through a prompt change and costs far more per call.
Streaming moderation	Deciding, as a response is generated, whether to keep sending it. The text released before a verdict has already reached the user.
Sliding window	Scoring only the most recent part of the text, so the cost of each check does not grow with the length of the response.
Taxonomy alignment	Whether the categories a model was trained on match the categories a test set or a product uses. A mismatch makes comparisons unfair.
Demographic bias	Scores that change with identity words alone, such as higher hate scores for a neutral sentence containing a group term.
Consent and opt-out	Using customer data for training only with agreement, and letting customers remove their data from training.
Annotator welfare	Protecting the people who read disturbing content: vetting, information about the risks, support resources and the right to stop.
Where to read	Markov et al.: A Holistic Approach to Undesired Content Detection in the Real World (arXiv:2208.03274, AAAI 2023)

14 Price the streaming cascade

DESIGN

SCENARIO

The paper scores finished text. You must moderate an LLM's response as it streams, using the paper's classifier as tier 1 and an LLM judge as tier 2 for uncertain cases. The paper describes none of this. Size the design before you propose it.

HOW TO THINK ABOUT IT

Cost the judge, then cost tier 1 under different checking policies. Do not skip tier 1.

Q What does the judge cost?

A The judge sees the response and a rubric: 700 input and 20 output tokens (assumed), at an assumed 3 dollars per million input and 15 per million output tokens: 0.0024 dollars a call.

- 50,000 responses a day: judging every one costs 120 dollars. Judging the 8% that are uncertain costs 9.60 dollars.

Q What does tier 1 cost?

A It depends on how often it runs and over how much text. Re-scoring all the text so far every 3 tokens, on a 500-token response: 166 checks and about 41,600 tokens scored, 83 times the response length.

- At an assumed 1/100 of the judge's per-token price, that is about 62 dollars a day over 50,000 responses.

Q What does the whole cascade cost?

A Whole text every 3 tokens: 62.37 + 9.60 = 71.97 dollars, a 40% saving. Last 64 tokens every 3 tokens: 25.54 dollars, 79%. Whole text every 50 tokens: 13.73 dollars, 89%. One pass at the end: 10.35 dollars, 91%.

- The cascade only pays if tier 1 is cheap across the whole stream.

Q What is the trade-off in checking less often?

A More text is released before a verdict. Holding chunks before release delays the text, and releasing at once leaves flagged text visible.

- It is the same trade-off as moderating before or after publishing a document.

Q What is the cascade's ceiling?

A Tier 1's misses outside the uncertain band. Content scored confidently safe never reaches the judge, however good the judge is.

- Set thresholds per category from the precision-recall curve, and audit what tier 1 passes.

Q Reframe it.

A Cost every stage, including the cheap one, under the real access pattern. A cheap model run very many times is not cheap.

Basis: The cascade is this document's design and the paper does not describe it; the prices, 700 tokens, 8% and the 1/100 ratio are illustrative; arithmetic recomputed.

15 Review the stream gate

CODE REVIEW

SCENARIO

A colleague writes the stream gate below. In testing, flagged text still appears on screen before it is removed, some escalations seem to do nothing, and long responses get slow. Find the faults.

```
def moderate_stream(chunks, score_fn, judge_fn, low=0.3, high=0.7):
    text = ""
    for chunk in chunks:
        text += chunk
        score = score_fn(text)
        yield chunk                      # release immediately
        if score > high:
            yield "[removed]"
            return
        if score > low:
            judge_fn(text)               # escalate
```

HOW TO THINK ABOUT IT

Follow one flagged chunk through the function, then count how much text each check scores.

Q When is text released?

A The chunk is yielded before the score is acted on. A flagged chunk has reached the user by the time the removal message is sent.

- Hold a few chunks, decide, and then release or cut.

Q What does the judge call do?

A Nothing. In the uncertain band it calls the judge and ignores the answer, so escalations cannot change the outcome.

- Use its verdict to cut or release.

Q What does each check cost?

A It scores the whole text so far, so the cost of a response grows with the square of its length (Question 14).

- Score a window of recent text, or check every few tokens.

Q What is missing at the end?

A A final check on the text still held when the stream ends, so the tail of a response is never released unchecked.

- Also decide what the user sees after a cut, and log it for audit.

Q State the fix, and a test.

A Hold chunks, score a window, act on the judge's verdict, release held chunks only after a pass, and check the tail at the end.

- Test: a violating sentence in the middle never appears in the output, and a long clean response is scored in linear time.

Q Reframe it.

A In a stream, the order of release and decision is the control. Check that nothing leaves before the check that could stop it.

REFERENCE CODE

```
def moderate_stream(chunks, score_fn, judge_fn, low=0.3, high=0.7, hold=3, window=400):
    held, text = [], ""
    for chunk in chunks:
        held.append(chunk)
        text += chunk
        if len(held) < hold:
            continue
        recent = text[-window:]                      # score a window, not everything
        score = score_fn(recent)
        if score >= high or (score >= low and judge_fn(recent)):
            yield "[response removed: policy violation]"
            return
        yield from held
        held = []
    if held:                                         # check the tail before release
        tail = score_fn(text[-window:])
        if tail >= high or (tail >= low and judge_fn(text[-window:])):
            yield "[response removed: policy violation]"
            return
    yield from held
```

Basis: The design is this document's and the paper has no streaming discussion; the faults and the fix are reasoned, and the fix was checked by hand and not run.

16 Read the comparison table honestly

EVALUATION

SCENARIO

A vendor slide says: our model beats Perspective API everywhere. The paper's Table 3 compares the two. Say what the table supports and what it does not.

HOW TO THINK ABOUT IT

List where each side wins, and read the table's own notes on the test sets and the starred cells.

Q Where does the paper's model win?

A On the public test set labeled with its own taxonomy: S .9703 against .8709, H .7968 against .6914, V .7371 against .5201, HR .6191 against .3902. On TweetEval hate (.6473 against .5961), Stormfront hate (.9053 against .8754), Jigsaw identity hate (.6890 against .6644) and Reddit sexual (.9417 against .8961) (source-confirmed).

- Perspective API does not explicitly cover SH, S3, H2 and V2, so there is nothing to compare there.

Q Where does Perspective API win?

A On Jigsaw insult (.8814 against .8548), obscene (.9500 against .8353), threat (.7492 against .6144) and toxic (.9769 against .9304), and on TweetEval offensive (.7919 against .7024).

- Those categories fit its own taxonomy. The paper says it is not surprising, and that it shows how important it is to align the taxonomy between training data and evaluation.

Q What do the stars mean?

A Cells with a star come from an approximate taxonomy match, so the paper says they are not an exact fair comparison. Several of the cells where Perspective API wins are starred for the paper's model.

- Do not state a starred difference as exact.

Q What is missing?

A A production result. The production test set cannot be shared, so Table 3 uses public data only.

- Also, AUPRC depends on each test set's prevalence, so numbers from different sets cannot be ranked against each other.

Q Reframe it.

A A comparison is only as fair as the match between the test set and each system's design. Report the losses next to the wins, and say what was not tested.

Basis: All Table 3 values and the paper's notes are from the paper (source-confirmed); the point about prevalence is standard.

17 What does the paper say its model gets wrong?

LIMITS

SCENARIO

A launch review asks for the known limits of the classifier. The paper has a section on them. List them and say how you would monitor each one.

HOW TO THINK ABOUT IT

Collect the limits the paper states, then attach a measurement to each.

Q What are the stated limits?

A The data is US-centric and the taxonomy was designed to best fit it. Harassment was not yet available in the public endpoint. Only about 5% of training samples are non-English, and multilingual performance was not rigorously evaluated (source-confirmed).

- Red-teaming is internal, done with each new model version, which the paper calls not scalable.

Q What about bias?

A The model may give higher hate scores to input containing gay and higher sexual scores to input containing her. Template data did not fully remove it. The paper adds disproportionate false positives when discussing groups that are often targets of hate.

- It is a harm to the people the policy is meant to protect.

Q How would you monitor?

A Per-category scores on fresh production samples, scores on paired templates that differ only in an identity word, key token probing repeated on a schedule, and performance on each language you serve.

- Track the share of key tokens that are irrelevant symbols or near copies of training text.

Q What would you not claim?

A That the model is fair, or accurate in every language, or complete. The paper says there is no silver bullet and recommends using the taxonomy with other mitigations.

- Say which languages and contexts were measured.

Q Reframe it.

A Turn each stated limit into something you measure on a schedule. A known limit is only useful if someone watches it.

Basis: The limits, the bias examples and the no-silver-bullet statement are from the paper (source-confirmed); the monitoring plan is reasoned.

18 Data consent, annotator welfare and ten times the traffic

OPERATIONS

SCENARIO

Your company wants to train a moderation model on customer traffic and expects ten times the volume next year. List the obligations and what scales.

HOW TO THINK ABOUT IT

Use the paper's statements on consent, privacy and annotators, then scale each quantity by ten.

Q What does the paper say about data use?

A It obtains consent from all customers whose data is used to train its moderation models, lets customers opt out, includes no production data in the dataset it releases, masks personally identifiable information before scoring and labeling, and protects the labeling and active-learning pipelines with security controls (source-confirmed).

- It considers the risk of training data leakage extremely low, since the deployed model only computes scores and cannot generate text.

Q What does it say about annotators?

A Annotators are selected for skill and willingness, vetted by counselors and told of the risks before opting in. Vendors provide mental health and wellness resources, and annotators can opt out at any point.

- Labeling at volume means reading disturbing text. Welfare is a cost to plan for.

Q What scales with ten times the traffic?

A Positives for rare labels: 4,000 self-harm cases a day at an assumed 10 million submissions become 40,000 at 100 million. Scoring cost, one pass per item. Streaming checks. Audit and drift work.

- The rates stay the same, and so does the labeling budget unless it is raised.

Q What does not scale?

A The labeling cost per positive, which depends on the rate and the selection method. Active learning keeps it near 5,400 labels per 100 positives for self-harm, against 111,000 at random.

- More traffic gives the selection step more to choose from, which can improve its yield.

Q Reframe it.

A Plan for the people and the data governance as well as for the machines. Some costs follow volume, and some follow the method.

Basis: The consent, privacy and welfare statements are from the paper (source-confirmed); the 10 million and 100 million submissions are illustrative; arithmetic recomputed.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #5 / #11. The one source is the OpenAI paper (arXiv:2208.03274, published at AAAI 2023), whose full text was read on 2026-10-17. The paper reports results on a public test set and gives no traffic volume, latency or throughput. It says nothing about streaming moderation or a cascade of models, so the answers about those are design arguments made for this book and were not run on real systems.

- Markov, Zhang, Agarwal, Eloundou, Lee, Adler, Jiang, Weng: A Holistic Approach to Undesired Content Detection in the Real World, arXiv:2208.03274 (v2, February 2023), AAAI 2023, full text read 2026-10-17
https://arxiv.org/abs/2208.03274